```
"""
fundamental.py
--------------
Computes a composite "fundamental strength score" for each currency from
macro data you supply (see fundamental_data_template.csv).

Method:
  1. Each metric is z-scored across the currency set you provide, so a
     currency's score reflects how it compares to the others in the basket
     -- not some fixed absolute threshold.
  2. Metrics where "lower is stronger" (e.g. unemployment) are inverted
     before z-scoring.
  3. A weighted sum of the z-scores gives the composite strength score.
     Weights are editable in WEIGHTS below -- tune them to reflect which
     drivers you think matter most right now.

Composite score is unitless. What matters is the *rank*, and the *spread*
between currencies (a big gap = a more confident fundamental bias).
"""

import pandas as pd
import numpy as np

# Metric -> weight in the composite score. Positive weight = higher value is
# bullish for the currency. Adjust freely.
WEIGHTS = {
    "InterestRate": 1.5,       # higher policy rate -> typically supportive
    "RateBias": 1.5,           # hawkish forward guidance -> supportive
    "CPI_YoY": 0.5,            # mild positive weight: inflation w/ hawkish bank
    "GDP_QoQ": 1.0,            # stronger growth -> supportive
    "Unemployment": -1.0,      # inverted: lower unemployment -> supportive
    "PMI_Manufacturing": 1.0,  # >50 = expansion
    "PMI_Services": 1.0,
    "TradeBalance_Bn": 0.5,    # surplus -> mild support
    "RetailSales_MoM": 0.5,
}


def load_fundamental_data(csv_path: str) -> pd.DataFrame:
    df = pd.read_csv(csv_path)
    if "Currency" not in df.columns:
        raise ValueError("CSV must have a 'Currency' column.")
    df = df.set_index("Currency")
    # Any metric column not present at all gets added as all-NaN, so a
```

```
    # blank/missing metric (e.g. PMI when auto-fetched) is treated as
    # "no data" rather than a hard error -- it's excluded from that
    # currency's score instead of being penalized.
    for metric in WEIGHTS:
        if metric not in df.columns:
            df[metric] = np.nan
    return df


def zscore(series: pd.Series, clip: float = 2.5) -> pd.Series:
    """
    Z-score with clipping (default +/-2.5 std). Clipping matters here: a
    single extreme outlier (e.g. a currency in a high-inflation regime like
    TRY) can otherwise dominate the composite score out of proportion to
    its actual weight. Clipping caps that influence while preserving rank.
    """
    std = series.std(ddof=0)
    if std == 0 or np.isnan(std):
        return pd.Series(0.0, index=series.index)
    z = (series - series.mean()) / std
    return z.clip(lower=-clip, upper=clip)


def compute_currency_strength(df: pd.DataFrame) -> pd.DataFrame:
    """
    Returns a DataFrame indexed by currency with each metric's z-score,
    a 'CompositeScore', and 'Rank' (1 = strongest).
    """
    z = pd.DataFrame(index=df.index)
    for metric, weight in WEIGHTS.items():
        col_z = zscore(df[metric])
        if weight < 0:
            col_z = -col_z
        z[metric + "_z"] = col_z * abs(weight)

    z["CompositeScore"] = z.sum(axis=1)
    z["Rank"] = z["CompositeScore"].rank(ascending=False).astype(int)
    z = z.sort_values("CompositeScore", ascending=False)
    return z


def strength_table(csv_path: str) -> pd.DataFrame:
    df = load_fundamental_data(csv_path)
    scored = compute_currency_strength(df)
    return scored[["CompositeScore", "Rank"]]
```

```
if __name__ == "__main__":
    table = strength_table("fundamental_data_template.csv")
    print("=== Currency Strength Ranking (fundamental) ===")
    print(table.to_string(float_format=lambda x: f"{x:.2f}"))
```